ChatGPT

MVP Development Documentation: Local Speech→Speech Translation on RTX A4000

Objective

Build an **end-to-end speech→speech translation MVP** that runs on a local Windows machine equipped with an **NVIDIA RTX A4000 (16 GB GDDR6 memory)**. The pipeline should capture live microphone audio, translate it into a target language and speak it back in the original speaker's voice with **<2 s latency**. This document outlines the hardware requirements, environment setup, model selection, and implementation steps.

1. Hardware and System Requirements

Component	Requirement	Evidence
GPU	NVIDIA RTX A4000 with at least 6 GB free GPU memory. The A4000 is a professional Ampere-family GPU with **16 GB GDDR6 ECC memory** [1] and can run the Canary speech-translation model (requires ≈6 GB to load) [2].	NVIDIA product page lists the A4000 memory and other specs [1].
CPU & RAM	Modern x64 CPU, ≥16 GB system RAM. ASR/AST models are GPU-bound, but adequate RAM is needed to feed audio buffers and run Python.	—
Operating System	While the machine runs Windows 10/11, **NeMo models only list Linux as the preferred OS** [3] [4]. For a Windows host, run the inference code inside **WSL2 Ubuntu** or a Linux virtual machine to ensure compatibility.	Model documentation states Linux is the preferred/ supported OS [3] [4].
Network	Internet connection to download models and call the **ElevenLabs TTS API**. Use TLS/HTTPS to protect data.	—
Audio IO	A microphone for input and a virtual audio device (e.g., **VB-Audio Virtual Cable**) to send translated audio to OBS/ streaming software.	—

Why WSL2? NVIDIA's Canary and Parakeet models specify Linux as the supported OS [3] [4]. Running them inside WSL2 (with GPU passthrough via CUDA on WSL) ensures the GPU is accessible. Install the latest **NVIDIA GPU driver** for Windows and the matching **CUDA toolkit** inside WSL2.

2. Environment Setup

1. **Install NVIDIA Drivers & CUDA**
2. Install the latest **NVIDIA RTX A4000 driver** on Windows. Enable CUDA on WSL (requires Windows 11 build 21H2+). NVIDIA's CUDA 11/12 drivers for WSL provide GPU acceleration inside Ubuntu.
3. **Install WSL2 + Ubuntu**
4. From a PowerShell terminal run:

```
wsl --install -d Ubuntu-22.04
```

5. Ensure `wsl --update` and `wsl --shutdown` to get the latest kernel.
6. **Inside WSL: create a Python environment**

```
sudo apt update && sudo apt install python3.10 python3.10-venv python3.10-
dev ffmpeg -y
python3.10 -m venv ~/speechenv
source ~/speechenv/bin/activate
pip install --upgrade pip
```

7. **Install PyTorch with CUDA**

```
pip install --extra-index-url https://download.pytorch.org/whl/cu121
torch==2.2.0+cu121 torchaudio==2.2.0+cu121 torchvision
```

8. **Install NeMo and audio dependencies** The Canary and Parakeet models require the **main branch of NeMo**. Install it along with audio libraries:

```
pip install git+https://github.com/NVIDIA/NeMo.git@main
pip install sounddevice websockets numpy scipy
```

   The `sounddevice` library captures microphone audio; `websockets` supports streaming to/from a client if needed.

9. **Install VB-Audio Virtual Cable (Windows)**
10. Download from `https://vb-audio.com/Cable/index.htm` and install. It creates a virtual input/output device. Set this as the **output device** for the translation script so OBS or any streaming software can pick up the translated voice.

3. Model Selection

3.1 Canary-1b-v2 (Speech Translation)

- **Functionality:** A **1-billion-parameter** model for **speech transcription and translation** across **25 European languages** [5], supporting English→X and X→English. It produces transcribed text or direct translation (AST) with automatic punctuation and capitalization [6].
- **Hardware & OS:** Requires ~6 GB of GPU memory and is supported on NVIDIA Ampere GPUs; Linux is the preferred OS [2].
- **Languages:** Bulgarian, Croatian, Czech, Danish, Dutch, English, Estonian, Finnish, French, German, Greek, Hungarian, Italian, Latvian, Lithuanian, Maltese, Polish, Portuguese, Romanian, Slovak, Slovenian, Spanish, Swedish, Russian and Ukrainian [7].
- **Installation:** Load the model with NeMo's `ASRModel.from_pretrained()` API:

```
import nemo.collections.asr as nemo_asr
asr_ast_model =
nemo_asr.models.ASRModel.from_pretrained(model_name="nvidia/canary-1b-v2")
# Example translation (English → Russian)
output = asr_ast_model.transcribe(['audio.wav'], source_lang='en',
target_lang='ru')
print(output[0].text)
```

3.2 Parakeet-tdt-0.6b-v3 (ASR – fallback)

- **Functionality:** A **600 M-parameter ASR model** optimized for high throughput and multilingual transcription. It auto-detects input language and supports the same 25 languages [8]. It is ideal for generating source-language captions or as a fallback if you want to run a separate translation system.
- **Hardware & OS:** Needs ~2 GB GPU memory and prefers Linux [9].

3.3 ElevenLabs TTS

- **Purpose:** Convert translated text back to speech while preserving the speaker's voice. The **Flash v2.5** model offers **~75 ms latency** across 32 languages [10] and is optimized for real-time applications.
- **Voice cloning:** ElevenLabs supports **Instant Voice Cloning**; you upload a **60-second audio sample** to create a custom voice [11]. The company verifies the audio and delivers a clone almost immediately [12].
- **Usage:** The API call is made over HTTPS and returns audio bytes. Provide your API key in the header and specify the cloned `voice_id` and the `eleven_flash_v2_5` model.

4. Implementation Steps

4.1 Capture Audio

1. Use the **sounddevice** library to record microphone audio at 16 kHz (mono). Choose a **chunk size of 0.5–1 s** to balance latency and context.

2. Optionally, integrate a **Voice Activity Detector (VAD)** (e.g., `webrtcvad` or `silero-vad`) to detect speech segments. This improves translation quality by ensuring you send complete clauses.
3. For streaming from other apps (e.g., RTMP feed), capture the audio and feed it to the same pipeline.

```
import sounddevice as sd
import numpy as np

def record_chunks(chunk_secs=0.5, samplerate=16000):
    chunk_samples = int(chunk_secs * samplerate)
    while True:
        audio = sd.rec(frames=chunk_samples, samplerate=samplerate, channels=1,
dtype='float32')
        sd.wait()
        yield audio.flatten()
```

4.2 Translation with Canary

1. **Instantiate the model** (once) as shown above. Set `source_lang` and `target_lang` when calling `.transcribe()`.
2. **Process chunks:** For each audio chunk, convert the NumPy array to a temporary `.wav` or directly pass the array (NeMo currently expects file paths; you may need to save to a `BytesIO` stream or small temporary file).

```
from scipy.io.wavfile import write
import tempfile

for chunk in record_chunks():
    with tempfile.NamedTemporaryFile(suffix='.wav', delete=True) as tmp:
        write(tmp.name, 16000, (chunk * 32767).astype(np.int16))
        result = asr_ast_model.transcribe([tmp.name], source_lang='ru',
target_lang='en')
        translated_text = result[0].text
        # Send translated_text to TTS
```

Pipelining: Because translation takes time, pipeline the processing: while chunk *N* is being captured, chunk *N–1* is being translated and chunk *N–2* is being synthesized. This overlapping reduces end-to-end latency.

4.3 Text-to-Speech with ElevenLabs

1. **Prepare the TTS request:** Use your `voice_id` from voice cloning and choose the `eleven_flash_v2_5` model for low latency [10].
2. **Make HTTPS request:**

```
import requests

def tts_elevenlabs(text, voice_id, api_key):
    url = f"https://api.elevenlabs.io/v1/text-to-speech/{voice_id}"
    headers = {
        'xi-api-key': api_key,
        'Content-Type': 'application/json'
    }
    payload = {
        'model_id': 'eleven_flash_v2_5',
        'text': text,
        'voice_settings': {
            'stability': 0.5,
            'similarity_boost': 0.8
        }
    }
    resp = requests.post(url, json=payload, headers=headers)
    resp.raise_for_status()
    return resp.content  # returns audio bytes (e.g., MP3)
```

3. **Play audio:** Use `sounddevice` or `pydub` to stream audio to the virtual cable.

```
from pydub import AudioSegment
import io, numpy as np

def play_audio(audio_bytes):
    audio = AudioSegment.from_file(io.BytesIO(audio_bytes), format='mp3')
    samples = np.array(audio.get_array_of_samples()).astype(np.float32) /
32768.0
    sd.play(samples, samplerate=audio.frame_rate, device='VB-Cable')
    sd.wait()
```

4.4 Integration with OBS/Streaming

1. **OBS Setup:** In OBS, add your microphone as the primary audio source. Add a **new audio input capture** using the **VB-Audio Virtual Cable**; this will carry the translated voice.
2. Adjust the volume of the translated audio relative to the original to avoid echo. You may need to add a small delay filter to synchronize with the video feed (start with ~500 ms and adjust based on measured latency).
3. Optionally display a small latency meter in your script by time-stamping each chunk when captured and comparing it to when the audio finishes playing.

4.5 Error Handling and Fallbacks

- **Low Confidence or Missing Text:** If the translation result is empty or the model confidence is low, skip or request re-transcription. NeMo's API returns probabilities you can inspect.
- **ASR Fallback:** For critical segments (e.g., names), run the same audio through **Parakeet ASR** to obtain the source-language transcript, then use a translation API (e.g., DeepL). This ensures proper names are rendered correctly and provides subtitles for the source language.
- **Network Issues:** Implement retries for TTS API calls. If the TTS call fails, skip or replay the last output.

4.6 Data Privacy

- All communications with ElevenLabs should use HTTPS. Do not store raw audio or transcripts unless necessary for debugging. If logs are needed, sanitize them or store only metadata (e.g., timestamps, chunk IDs). Record the user's **consent for voice cloning** and keep a timestamped record of the agreement.

5. Summary and Recommendations

- The **Canary-1b-v2** model offers an all-in-one **speech translation** solution across 25 European languages [5] and can run on the RTX A4000 when launched under Linux/WSL2. It produces both transcriptions and translated text, eliminating the need for a separate ASR model for the quick speech→speech path.
- Combine **Canary** with **ElevenLabs Flash v2.5** to achieve end-to-end speech→speech translation within roughly **1–2 seconds**; Flash v2.5's ultra-low latency (~75 ms) ensures the TTS stage is not the bottleneck [10].
- For voice authenticity, require users to upload a **60-second voice sample** to ElevenLabs; Instant Voice Cloning will generate a voice ID for the TTS API [11].
- Run the system as a **pipelined asynchronous loop**: capture audio → translate with Canary → synthesize with ElevenLabs → play audio. Overlap each stage to minimize perceived delay.
- Keep the setup modular. If quality issues arise or you need subtitles, run **Parakeet** for ASR and call an MT API (e.g., DeepL) separately.

By following the steps above, a solo developer can assemble a functional MVP on a local Windows workstation with an RTX A4000 GPU. The combination of NeMo's state-of-the-art speech translation and ElevenLabs' low-latency voice cloning enables near-real-time translation while preserving speaker identity—an essential requirement for engaging multilingual streams.

---

[1] RTX A4000 Graphics Card | NVIDIA
https://www.nvidia.com/en-us/products/workstations/rtx-a4000/

[2] [3] [5] [6] [7] nvidia/canary-1b-v2 · Hugging Face
https://huggingface.co/nvidia/canary-1b-v2

[4] [8] [9] nvidia/parakeet-tdt-0.6b-v3 · Hugging Face
https://huggingface.co/nvidia/parakeet-tdt-0.6b-v3

10 Models | ElevenLabs Documentation
https://elevenlabs.io/docs/models

11 12 ElevenLabs — How to Clone Your Voice in 2025 (Guide) | ElevenLabs
https://elevenlabs.io/blog/how-to-clone-voice